

Deanonymizing Tor Onion Services Through Apache Server-Status Pages

Velyan Kolev
CFLW Cyber Strategies
The Hague, The Netherlands
velyan.kolev@cflw.com

Mark van Staalduinen
CFLW Cyber Strategies
The Hague, The Netherlands
mark.vanstaalduinen@cflw.com

Georgios Smaragdakis
Delft University of Technology
Delft, The Netherlands
g.smaragdakis@tudelft.nl

Abstract

Tor onion services are designed to conceal the network location of their hosting infrastructure, which makes attribution difficult. However, Apache server-status pages can unintentionally expose infrastructure information when onion-service traffic is forwarded to a local web server. In this configuration, Apache may treat requests arriving through Tor as localhost traffic and allow access to a page intended to be restricted to the local machine. We analyze 210,121 validated server-status snapshots from 12,591 onion services collected between November 2019 and March 2025. We show how fields such as `via`, `VHost`, and the `Workers Table` can reveal public IP addresses, provider-specific hostnames, and other infrastructure leads. Repeated observations show that this exposure persisted throughout the measurement period, with hundreds to thousands of affected onion services active at a time. We also present a real-world case in which an exposed VPS hostname provided a key technical lead to the hosting infrastructure of two large child sexual abuse material platforms. Follow-up by German law enforcement contributed to the identification of the alleged operator and the takedown of both services. These findings show that exposed server-status pages provide a passive source of infrastructure attribution leads without attacking the Tor protocol itself.

CCS Concepts

• Security and privacy → Privacy-preserving protocols.

Keywords

Tor onion services, Apache `mod_status`, virtual hosts, infrastructure attribution

ACM Reference Format:

Velyan Kolev, Mark van Staalduinen, and Georgios Smaragdakis. 2026. Deanonymizing Tor Onion Services Through Apache Server-Status Pages. In *1st Workshop on Technical Analysis and Knowledge Exchange on Disrupting Online Criminal Networks (TAKEDOWN '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3847159.3848614>

1 Introduction

Tor onion services were created to provide privacy-preserving and censorship-resistant access to online services by hiding the location of the server behind an onion address [25]. They can be used

by journalists, activists, whistleblowers, and people living under authoritative or restrictive regimes. However, onion services are also known to be used for criminal activity, including illicit marketplaces [20], ransomware infrastructure, fraud, and child sexual abuse material (CSAM) platforms [9].

This creates a challenge for law-enforcement agencies and other investigators, as Tor is specifically designed so that visitors cannot directly determine where an onion service is hosted. Yet, when an onion service is used for criminal activity, identifying the server or hosting provider behind it can be important for an investigation. Recent public takedown operations show that this remains a relevant practical problem [6]. Therefore, there is great value in technical methods that can reveal useful infrastructure information from a service itself, as long as the information is interpreted carefully and not treated as automatic proof of who operates the service [18, 26].

Although onion services hide their server location from visitors, they still rely on standard web-server software, such as Apache HTTP Server [1] and nginx [19], to serve web content. Like any other web service, this infrastructure can be misconfigured and unintentionally expose information about the server or its configuration. This paper examines one such misconfiguration in Apache server's `mod_status` diagnostic interface. The interface is commonly available at `/server-status` and can reveal information about the server, current requests, and configured virtual hosts.

Administrators often restrict this page to localhost, expecting that external visitors cannot access it. However, an onion service can forward a visitor's request to a local Apache backend. Apache then sees the final connection as coming from localhost and may allow access to the diagnostic page. The issue is therefore not a weakness in Tor or Apache; it results from the way Tor forwards requests to backend servers and how Apache evaluates source-IP-based access rules.

Previous work and operational-security guidance have warned that services restricted to localhost can still be reachable through onion services [11, 12, 21, 23]. However, there has been limited analysis of how often Apache server-status pages are exposed in practice, whether this exposure persists over time, and what information from these pages can actually be used in an investigation.

To address this gap, we conduct a large-scale, multi-year analysis of Apache server-status exposure across Tor onion services. We analyze 210,121 snapshots from 12,591 onion services collected by the CFLW Dark Web Monitor [7] between November 2019 and March 2025. Our results show that the problem is not limited to isolated misconfigurations; server-status exposure persisted throughout the observation period, with hundreds to thousands of pages estimated to be active at a time. We also show how exposed fields can provide supporting context during an investigation.



This work is licensed under a Creative Commons Attribution 4.0 International License. *TAKEDOWN '26, The Hague, Netherlands*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3020-7/2026/11
<https://doi.org/10.1145/3847159.3848614>

We illustrate what this can mean in practice through a 2025 takedown case involving two darknet CSAM video platforms. In this case, a provider-specific hostname exposed in an Apache server-status page helped investigators identify the relevant hosting provider and supported the investigation, which ultimately led to the seizure of the servers and the arrest of the alleged operator. This paper makes four contributions:

- We explain and experimentally demonstrate how Tor backend forwarding can make a localhost-only server-status restriction ineffective.
- We measure the persistence of exposed Apache server-status pages across more than five years of observations.
- We show how different fields on a server-status page can reveal direct technical infrastructure information, identify a relevant hosted asset, and support active validation of suspected infrastructure relationships.
- We present a case study showing how a hostname exposed in a virtual-host field provided the key technical detail in the investigation and takedown of two CSAM platforms.

The remainder of this paper is organized as follows. Section 2 introduces Tor onion-service forwarding and the relevant Apache concepts. Section 3 explains the localhost pitfall. Section 4 presents the dataset, analysis method, and results. Section 5 describes the takedown case, while Section 6 discusses the implications, limitations, and future work. Finally, Section 7 reviews related research, and Section 8 concludes the paper.

2 Background: Tor and Apache

The contributions within this paper depend on understanding two concepts: how Tor Onion Services forwards traffic to backend servers and how Apache HTTP Server is typically set up to serve web content. This section covers both of these topics as well as a short section on Virtual Hosts, the mechanism used by an Apache server to host multiple websites on the same backend server. Figure 1 shows how multiple communication flows work in tandem to serve a website on Tor using Apache while restricting sensitive server data to unauthorized public-facing users.

2.1 Onion Services and Backend Forwarding

Onion services, formerly known as "hidden services", are websites that are only accessible through the Tor network. An onion service makes a web server available through a .onion address without revealing the network location of that server to its visitors.

As shown in steps 1–2 of Figure 1, the Tor process receives traffic for the onion service and forwards it to a web server chosen by the operator. In a typical setup, Tor and Apache run on the same machine, and Apache listens only on a local address or port. This design keeps the web server behind Tor while still allowing it to serve normal HTTP content.

Forwarding behavior is configured with Tor's `HiddenServicePort`. For example, "`HiddenServicePort 80 127.0.0.1:9090`" maps port 80 of an onion service to an Apache instance listening on 127.0.0.1:9090 [24]. The result is a Tor process forwarding an onion-service request to this local endpoint, and Apache returning the response through Tor. Apache therefore receives the connection

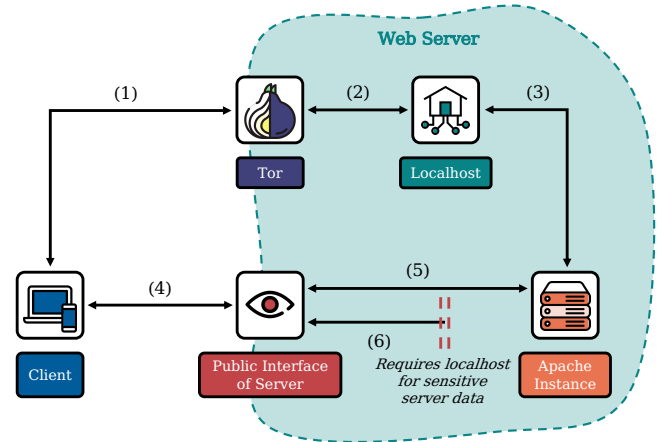


Figure 1: Request paths relevant to the localhost pitfall. Tor receives onion-service traffic (step 1) and forwards it to Apache through localhost (step 2). A client normally reaches Apache through the public interface (steps 4–5), while local administration uses localhost (step 3). A localhost-only rule blocks public requests from accessing sensitive endpoints (step 6).

from the local Tor process rather than directly from the remote visitor.

2.2 Apache `mod_status` and Localhost Restrictions

Apache's `mod_status` module provides a server-status page, normally available at `/server-status`, for administrators to inspect the current state of an Apache instance [2]. Depending on the version and configuration, the page can show the server version, uptime, load statistics, active connections, recent requests, and virtual-host configurations. An example of an Apache server-status page can be seen in Figure 2. Sensitive infrastructure values, including hostnames and onion addresses, have been redacted.

While this information is useful for administration and debugging, it can also expose details about the server and the services it hosts. Hence, operators commonly enable the page but restrict it to only be accessible by local access, e.g., with a directive such as `Require ip 127.0.0.1` [22]. This rule checks the source address of the connection received by Apache and rejects public access to the server-status page. Respectively, in Figure 1, a local administrator reaches Apache through localhost (step 3), while a client requesting the server's public interface uses the public route (steps 4–5). Under ordinary clearnet access, the rule works as expected: Apache allows viewing the server-status page in the local request and denies the public-interface request (step 6).

2.3 Virtual Hosts

A Virtual Host is the mechanism used by Apache to serve multiple websites or services from one instance [3]. Using so-called `VirtualHost` blocks, each block defines a separate `ServerName` and content, allowing clearnet or onion services to share the same backend while appearing independent to their users [4].

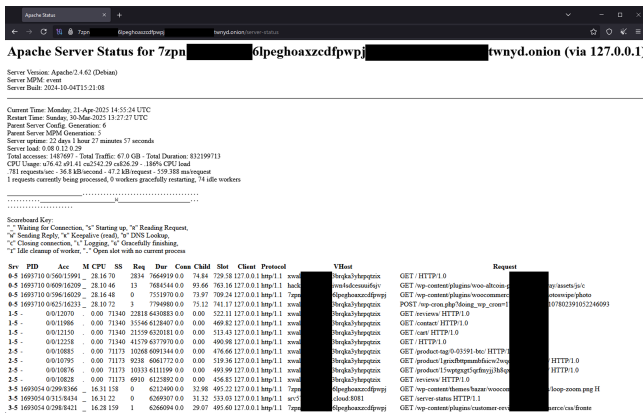


Figure 2: Example of an Apache /server-status page exposed on a dark web onion service. The page shows server properties, live server load statistics, virtual hosts, and metadata about recent connections. Sensitive infrastructure values have been redacted.

To show the correct website to a user, Apache matches an incoming request against all configured virtual hosts; the most relevant one is selected to serve the request. The selection procedure first finds the applicable address and port, then uses the HTTP Host header to select a matching virtual host. If no hostname matches, it falls back to the first matching virtual host.

If a server-status page is enabled, its Workers Table records recent requests and the virtual host that handled them. Note the VHost column in Figure 2. This table is global to the Apache instance itself. Therefore, any configured virtual host can be seen serving requests from a single Apache server-status page.

3 The Localhost Pitfall

The two configurations interact in an unexpected way. Tor’s forwarding connection to Apache follows the local path shown in step 2 of Figure 1, even though the original onion-service visitor is remote. Apache therefore applies the same localhost rule that it would apply to a request from a local administrator in step 3. A `Require ip 127.0.0.1` restriction can protect the page from direct clearnet access while still allowing access through the onion service.

The localhost pitfall requires four conditions to coincide, i.e., `mod_status` is enabled, `/server-status` is reachable through the Apache listener serving the onion service, the access rule trusts the loopback address, and Tor forwards onion-service traffic to that trusted loopback endpoint.

Figure 3 shows this behavior in a controlled comparison. The status page is accessible from localhost, Apache returns Forbidden when the page is requested through the server’s public IP, and the same page is accessible through the onion address without changing the configuration. Testing only the public interface can therefore make the restriction appear effective while leaving the status page exposed to onion-service visitors. Because the figure shows only our controlled test infrastructure, no values have been redacted.

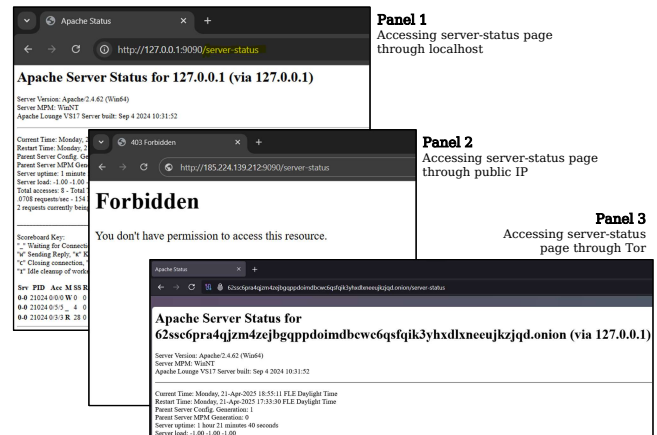


Figure 3: The server-status page is accessible from localhost (Panel 1), denied through the server’s public IP (Panel 2), and accessible again through the onion service (Panel 3). The test was conducted on infrastructure operated by the authors, so no values have been redacted.

Notably, this is a deployment and configuration issue, rather than a flaw in Tor or Apache. Tor forwards traffic to the endpoint selected by the operator, and Apache applies the configured source-IP rule to the connection it receives. The problem is using a localhost-only source-IP check as the only protection for a diagnostic page that is reachable through a Tor-facing backend route.

The forwarding behavior explains why the page becomes accessible. Its investigative value depends on the server metadata Apache exposes once the page is reached.

4 Analysis of Exposed Server-Status Pages

This section analyses exposed Apache server-status pages in five steps. First, we define the validated dataset of onion-service snapshots. Second, we explain how selected server-status fields are interpreted when Apache is deployed behind Tor. Third, we measure how persistently server-status pages were exposed over time. Fourth, we examine which fields provide direct infrastructure signals, which provide leads, and which should be treated only as supporting context. Finally, we show how the Workers Table can be used to actively validate a suspected infrastructure relationship.

4.1 Dataset

This study uses a curated dataset of repeated observations collected through the Dark Web Monitor (DWM), an open-source intelligence crawler developed by CFLW Cyber Strategies [7]. The DWM continuously collects web content from anonymous networks, including Tor. As part of its extended crawling, it visits predefined paths that can indicate server misconfigurations, including `/server-status`. The DWM has collected this path since November 2019.

Candidate pages were identified from responses at `/server-status` using known Apache status-page titles and, for non-standard titles, checks for the expected Apache status-page content. We then excluded error responses, unrelated pages, stale or static copies, and other responses that did not represent a genuine server-status page.

The resulting validated dataset contains 12,591 onion services with exposed server-status pages and 210,121 snapshots. A snapshot represents the state of a page at a particular point in time and includes its raw HTML and extracted metadata.

The dataset covers November 2019 to March 2025. The repeated snapshots let us examine whether server-status pages persisted over time, rather than only whether they were observed once. For context, during the measurement period, the DWM observed 913,732 onion services that have been active at least once. Hence, we estimate that around 1.4% of all active onions contained an exposed server-status page at some point.

4.2 Field Interpretation

The page includes many sources of useful information. In this paper, we focus on the heading fields `endpoint` and `via`, and the Workers Table columns `Client`, `VHost`, and `Request`.

We parse the valid pages using their HTML structure to identify page sections and regular expressions to extract the relevant fields. We then analyse what these fields mean in the context of an Apache server deployed behind Tor. This interpretation is necessary because the same value can have a different meaning depending on whether it reflects Tor’s local forwarding, a public-facing interface, a virtual-host configuration, or recent request activity.

We interpret these values by the field in which they appear and validate them independently. For example, a public address in `via` may point directly to the server infrastructure, while a provider-specific hostname in `VHost` may reveal the hosting provider or hosted asset. Other fields generally provide supporting context rather than a direct infrastructure lead. Table 1 summarizes the fields extracted from each page and how we use them in the analysis.

4.3 Persistent Exposure

To estimate how many onion services exposed a server-status page over time, we use repeated snapshots of validated pages. The DWM collects `/server-status` during extended scans, which revisit hosts every four to eight weeks, with an average interval of approximately six weeks.

For each server-status page, we define an activity interval from its first to its last validated snapshot. These observations provide only an approximate start and end of the exposure, since the page may have become accessible before its first observation or remained accessible after its last. We therefore extend the interval by three weeks at each end, corresponding to half the average scan interval. This accounts for uncertainty around when the exposure began and ended and gives services observed only once a finite activity interval. It does not imply continuous accessibility between observations.

Figure 4 shows the estimated number of exposed pages active at each point in time. The solid line uses the left axis for this estimate. For context, the dashed line uses the right axis to show the cumulative percentage of all onion services observed active by the DWM over the measurement period.

Exposed server-status pages were observed throughout the full measurement period. The estimated number ranged from hundreds to thousands at a time and exceeded 3,000 at its peak. As the overall observed onion-service population continued to grow, newly

Table 1: Fields extracted from Apache server-status pages and their role in this paper.

Page location	Field	Role in this paper
Heading	<code>endpoint</code>	The HTTP Host value from the original request. An unexpected value can indicate a proxy, copied page, or other inconsistency and requires validation.
Heading	<code>via</code>	The address Apache believes the request came from. A public address is the strongest direct technical infrastructure signal; a local address is expected when Tor forwards through localhost.
Workers Table	<code>Client</code>	The address associated with a recent connection. Public values generally reflect visitors or scanners and provide supporting context only.
Workers Table	<code>VHost</code>	The virtual host selected for a request. It can reveal service names or provider-specific hostnames and provide an infrastructure lead.
Workers Table	<code>Request</code>	The HTTP request line handled by a worker. It may contain paths, probes, or user-controlled values; it provides supporting context and can, in select cases, be used to actively validate suspected infrastructure relationships.

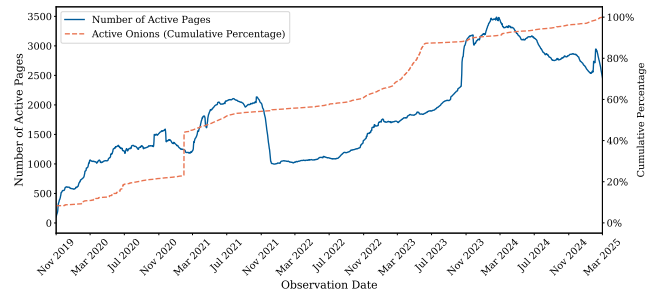


Figure 4: Estimated number of active onion services exposing Apache server-status over time (left axis) and cumulative percentage of the 913,732 DWM onion services observed active by each point in time (right axis). Activity intervals span the first to last validated snapshot with a three-week allowance at each end.

exposed server-status pages continued to appear rather than remaining limited to a fixed set of services discovered early in the dataset.

Because the estimate is based on intervals between positive observations, a page may have become unavailable and later reappeared between snapshots. The analysis can therefore overestimate continuous exposure, and the figure should be interpreted as an estimate of the broader persistence trend rather than continuous availability of individual pages.

Table 2: Public IP addresses by field. Occurrences count extracted public-IP values across all snapshots; a single snapshot may contribute multiple Workers Table values.

Field	Public occurrences	Unique public IPs	Interpretation
endpoint	0	0	Rare private-IP anomalies, likely caused by proxying or request rewriting.
via	713	43	Immediate technical infrastructure signal: the public-facing Apache interface used for the request.
Client	219,804	54,856	Visitor or scanner activity; supporting context only.
VHost	161,125	99	Backend or configuration context; hostname values can also provide a lead to a hosting provider.
Request	5,856	3,649	IPs appearing in request content, including probes or user-controlled input; supporting context only.

4.4 Infrastructure Leads from Exposed Fields

Server-status pages expose IP addresses and hostnames in several fields, but these values cannot be interpreted interchangeably. Their meaning depends on whether they describe the connection Apache received, the requested endpoint, a selected virtual host, or transient worker activity. We therefore distinguish an immediate technical infrastructure signal, a lead to a hosting provider or hosted asset, and supporting context that helps validate or interpret another lead. Table 2 summarizes the public-IP occurrences and unique public-IP counts observed in each field. It counts only IP-valued entries; hostname-valued VHost entries, including the provider-specific hostname discussed in Section 5, are excluded.

The heading contains the endpoint and via values. IP addresses in endpoint were rare: they appeared in 94 snapshots from nine pages, with only two unique private IPv4 addresses and no public addresses. These cases are consistent with proxying or request rewriting and do not provide an infrastructure lead. In contrast, via records the address Apache believes it received the request from. It appeared in 209,792 snapshots across 12,550 pages and was usually a local address, most often 127.0.0.1, as expected when Tor forwards traffic to a local backend. However, we observed 713 public-IP occurrences in via, representing 43 unique public addresses. A public via value is the strongest field-level result because it identifies the public-facing Apache interface used for that request. It remains a technical infrastructure lead, not proof of operator identity, ownership, or physical server location.

The VHost field can provide a different type of lead. It contained 161,125 public-IP occurrences, representing 99 unique public addresses. Such values can reflect an IP-based or fallback virtual-host configuration and provide context about the backend setup. Other VHost values are hostnames rather than IP addresses. In particular, a provider-specific hostname can identify a hosting provider or a concrete hosted asset and enable follow-up with that provider.

This requires validation and does not independently establish who operated the onion service or where the physical server was located. Section 5 illustrates this pathway in a takedown case.

The remaining Workers Table fields primarily provide supporting context. The Client field contained 219,804 public-IP occurrences and 54,856 unique public addresses, largely reflecting visitors, scanners, or other connections to the service. The Request field contained 5,856 public-IP occurrences and 3,649 unique public addresses, but these values occur inside request strings and can reflect probing, proxying, or user-controlled input. Together with selected VHost values, these fields can help validate recent activity and interpret other observations, but they are not standalone attribution evidence. The Request field can additionally support targeted active validation of a suspected infrastructure relationship, as described next.

4.5 Active Validation of Infrastructure Leads

Infrastructure values exposed on a server-status page do not always establish that a clearnet host or IP address is served by the same Apache instance as the onion service. In some cases, the Workers Table can be used to actively validate this relationship. Apache records recent HTTP request lines in the Request field, allowing an investigator to introduce a recognizable value and check whether it appears on the onion’s server-status page.

Figure 5 illustrates this process. First, a suspected clearnet host or IP address is identified, for example, from the VHost field. A request containing a unique query parameter is then sent to that endpoint. If the same value subsequently appears in the Request field of the onion service’s server-status page, this provides strong evidence that both endpoints are handled by the same Apache instance.

This technique provides a remote verification step for infrastructure leads that would otherwise remain uncertain. It requires both endpoints to be reachable and the request to remain visible in the Workers Table long enough to be observed.

5 Takedown of CSAM Video Platforms

In October 2025, German investigators dismantled two darknet video platforms, “Videos Sebick” and “Videos Yad,” that distributed child sexual abuse material (CSAM) [6, 8]. Before the takedown, the platforms had more than 350,000 registered users, hosted 21,324 videos with a combined duration of about 660 hours, and recorded nearly 900,000 video accesses. The investigation led to the seizure of servers in German data centres and, through international cooperation, the arrest of the alleged operator in Brazil. Figure 6 shows the seizure notice displayed on the darknet platforms after they were taken down.

The investigation that led to the takedown began with an exposed Apache server-status page. On 16 August 2023, “Videos Yad” exposed a server-status page whose VHost field contained a provider-specific virtual private server (VPS) hostname, shown in Figure 6. This was the initial technical lead in the case: the hostname revealed the relevant hosting provider and hosted asset, which allowed law enforcement to contact the provider and obtain further information. This follow-up formed the basis for identifying the alleged operator and ultimately seizing the servers. The same hosted

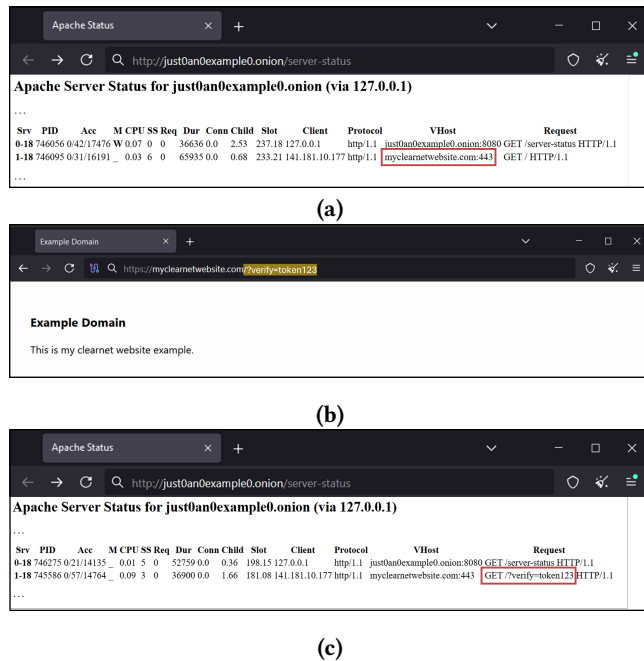


Figure 5: Active validation of shared Apache infrastructure. (a) A suspected clearnet endpoint is identified from the onion service’s server-status page. (b) The clearnet endpoint is requested with a unique query parameter. (c) The same value subsequently appears in the onion service’s Request field, providing evidence that both endpoints are handled by the same Apache instance.

asset was also exposed by the second onion service, providing an infrastructure link between the two platforms.

This case shows the practical impact of an exposed server-status page. The hostname did not itself identify the operator or prove the physical server location. However, it provided the starting point for the investigation and enabled the provider follow-up that supported the wider investigation and takedown.

6 Discussion

The central implication is not only that server-status pages remain exposed, but that a localhost-only rule can create a misleading security boundary in an onion-service deployment. It blocks the public interface an operator is likely to test, yet it does not distinguish a local administrator from Tor’s local forwarding connection. The persistence observed in the dataset is therefore consistent with a configuration pattern that can appear correctly restricted during routine administration. Although the dataset cannot establish why a particular operator left a page reachable, it shows that disabling the module is not the only relevant explanation: the protection may be deliberately enabled but incorrectly understood.

This distinction also shapes how exposed server-status data should be used. An exposed page should be treated as the start of an evidence chain rather than as a standalone deanonymization result. A public via address can provide a direct technical lead; a provider-specific VHost hostname can support a pivot to a hosting



Figure 6: Redacted technical lead in the CSAM takedown case: (a) shows the seizure notice displayed after the takedown; (b) shows the redacted Apache server-status VHost evidence: a provider-specific hostname that identified the relevant hosting provider and hosted asset. Sensitive infrastructure values have been redacted.

provider or hosted asset; and Workers Table values can corroborate the interpretation of those leads. Their value comes from how they guide validation and follow-up, not from any individual field proving operator identity.

In practice, this makes server-status collection most useful as a passive monitoring and infrastructure-enrichment capability. Investigators can extract via or VHost leads, validate them across repeated observations or other evidence, and use resulting hosting-provider or hosted-asset information for further investigation and provider follow-up. When the same infrastructure appears across multiple services, a single exposure may also reveal relationships that support a broader investigation. From a disruption perspective, such leads can turn an otherwise hidden onion service into an actionable infrastructure target, as demonstrated by the takedown case in Section 5.

6.1 Limitations and Mitigations

The method is opportunistic rather than targeted. An investigator cannot select an arbitrary onion service and expect it to expose useful information. Even when a server-status page is reachable, most fields do not provide a direct infrastructure lead. The persistence results are also interval-based estimates from repeated snapshots, rather than evidence of continuous exposure between every observation.

Interpretation and data quality impose further limits. Apache versions and configurations can change the structure or contents of a server-status page, so parsers require maintenance. Some values can be generic, transient, user-controlled, or affected by proxying. In particular, VHost and Workers Table values require validation and should not be treated as standalone attribution evidence.

The most reliable mitigation is to disable `mod_status` when it is not required. When operators need the page, they should expose it only through a separate management interface that Tor cannot reach and protect it with stronger access controls. A localhost-only source-IP rule is insufficient when Tor forwards onion-service traffic to that same local backend. Closing the page also cannot retract information that has already been collected.

6.2 Future Work

Future work could improve time-aware interpretation of repeated observations, assign confidence to leads based on the field and supporting evidence, and maintain parsers across Apache versions and layouts. It could also examine how multiple independent page fields and repeated snapshots can be combined to validate a lead without treating any single observation as conclusive.

7 Related Work

Prior work on onion-service deanonymization has used both network-layer and application-layer techniques. Network-layer approaches analyse properties of Tor traffic to infer the location of a hidden service. These techniques can be passive, such as traffic-correlation attacks, or active, such as circuit fingerprinting, but often require a privileged network position or interaction with the Tor network [5, 13, 17].

Application-layer approaches, on the other hand, examine information exposed by the service itself. One such work, CARONTE, scanned onion services for metadata including embedded IP addresses, analytics identifiers, reused TLS certificates, and clearnet domains, then used these observations to identify possible links to backend infrastructure [18]. Tippe et al. analysed real-world deanonymization cases and showed that server-side information leaks, including misconfigured headers, reused IP addresses, and internal hostnames, are frequently used as investigative leads [26].

Operational tools and reports have likewise identified server misconfigurations as a practical source of onion-service infrastructure information. OnionScan was designed to identify exposed directories, HTTP headers, clearnet links, tracking identifiers, and other server-side metadata [14]. Exposed Apache `mod_status` pages have also been noted as a source of co-hosted domains, client addresses, request paths, and Apache configuration details [11, 15, 16]. A 2021 report by SOS Intelligence described ransomware-related onion services whose exposed `mod_status` pages listed clearnet virtual

hosts; resolving and validating those values linked the services to backend hosting infrastructure [10].

However, prior work generally treats Apache server-status exposure as one of many possible misconfigurations. It does not systematically examine how persistently these pages are exposed or distinguish what individual fields can and cannot support as evidence when Apache is deployed behind Tor. This paper addresses that gap by analysing server-status pages as a structured source of server metadata in the context of Tor backend forwarding.

8 Conclusion

This paper examined how Apache `mod_status` pages can become accessible through Tor onion services despite a localhost-only access rule. Tor forwards onion-service requests to the local Apache backend, causing Apache to evaluate the connection as originating from localhost. Across 210,121 validated snapshots from 12,591 onion services collected between November 2019 and March 2025, this exposure persisted throughout the observation period, showing that it is not limited to isolated or short-lived configuration mistakes. These onion services accounted for around 1.4% of all active onion services in our dataset.

Exposed server-status pages can provide useful infrastructure leads, but their evidentiary value depends on the field in which a value appears. A public `via` address identifies the public-facing Apache interface used for a request, while a provider-specific VHost hostname can guide follow-up with a hosting provider or identify a hosted asset. The Workers Table can additionally support targeted active validation of suspected infrastructure relationships.

In the 2025 takedown case, the exposed VHost hostname was the starting point of the investigation: it revealed the relevant hosting provider and hosted asset and enabled the provider follow-up that supported the identification of the alleged operator and seizure of the servers. This demonstrates that such leads do not independently identify an operator, but can support a wider investigation when validated with other evidence. The exposure is not a weakness in Tor or Apache; it is a deployment problem. Operators should disable `mod_status` when it is unnecessary or isolate it from the backend interface reached through Tor.

Acknowledgments

We thank the anonymous reviewers for their constructive comments. This work has been partially supported by the European Union's Horizon 2020 and Horizon Europe Research and Innovation Programs under the projects ENSEMBLE (Grant Agreement #101168360) and SafeHorizon (Grant Agreement #101168562), and by the Digital Holland program, a Dutch national initiative uniting public and private organizations to drive digital research and innovation, focusing on key digital technologies and societal challenges, under the project AAPIE.

References

- [1] Apache. 2026. The Apache HTTP Server Project. <https://httpd.apache.org/>.
- [2] Apache Software Foundation. 2026. *Apache Module mod_status*. Retrieved July 12, 2026 from https://httpd.apache.org/docs/2.4/mod/mod_status.html
- [3] Apache Software Foundation. 2026. *Apache Virtual Host Documentation*. Retrieved July 12, 2026 from <https://httpd.apache.org/docs/2.4/vhosts/index.html>
- [4] Apache Software Foundation. 2026. *Virtual Host Examples*. Retrieved July 12, 2026 from <https://httpd.apache.org/docs/2.4/vhosts/examples.html>

- [5] Lamiaa Basyoni, Noora Fetais, Aiman Erbad, Amr Mohamed, and Mohsen Guizani. 2020. Traffic Analysis Attacks on Tor: A Survey. In *2020 IEEE International Conference on Informatics, IoT, and Enabling Technologies (ICIoT)*. IEEE, Doha, Qatar, 183–188. doi:10.1109/ICIoT48696.2020.9089497
- [6] Bayerisches Staatsministerium der Justiz. 2025. *Schlag gegen zwei der weltweit größten pädokriminellen Darknet-Plattformen*. Retrieved July 12, 2026 from <https://www.justiz.bayern.de/gerichte-und-behoerden/generalstaatsanwaltschaft/bamberg/presse/2025/25.php>
- [7] CFLW Cyber Strategies. 2020. *Dark Web Monitor*. Retrieved June 12, 2025 from <https://cflw.com/dwm/>
- [8] Die Bayerische Polizei. 2025. *Schlag gegen zwei der weltweit größten pädokriminellen Darknet-Plattformen*. Retrieved July 12, 2026 from <https://www.polizei.bayern.de/aktuelles/pressemitteilungen/092903/index.html>
- [9] Europol. 2024. *Internet Organised Crime Threat Assessment (IOCTA) 2024*. Publications Office of the European Union, Luxembourg.
- [10] Amir Hadzipasic. 2021. *A Special Investigation Exposing a Ransomware Group's Clear-Web IP and Their Duplicate Identities*. Retrieved June 25, 2025 from <https://sosintel.co.uk/a-special-investigation-exposing-a-ransomware-groups-clear-web-ip-and-their-duplicate-identities/>
- [11] Micah Hoffman. 2019. *Unhardened Web Servers in Tor Have No Anonymity*. Retrieved June 9, 2025 from https://www.osintcurio.us/2019/03/05/apache-mod_status-in-tor-hidden-services-destroy-anonymity/index.htm
- [12] Robert Koch. 2019. Hidden in the Shadow: The Dark Web—A Growing Risk for Military Operations?. In *2019 11th International Conference on Cyber Conflict (CyCon)*. IEEE, Tallinn, Estonia, 1–24. doi:10.23919/CYCON.2019.8756708
- [13] Albert Kwon, Mashael AlSabah, David Lazar, Marc Dacier, and Srinivas Devadas. 2015. Circuit Fingerprinting Attacks: Passive Deanonymization of Tor Hidden Services. In *24th USENIX Security Symposium (USENIX Security '15)*. USENIX Association, Washington, D.C., 287–302. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/kwon>
- [14] Sarah Jamie Lewis. 2016. *OnionScan: Investigating the Dark Web*. Retrieved June 24, 2025 from <https://onionscan.org/>
- [15] Sarah Jamie Lewis. 2016. *OnionScan Report: April 2016 – The Tor Network: Security and Crime*. Retrieved June 24, 2025 from <https://mascherari.press/onionscan-report-april-2016-the-tor-network-security-and-crime/>
- [16] Sarah Jamie Lewis. 2016. *OnionScan Report June 2016 – Snapshots of the Dark Web*. Retrieved June 24, 2025 from <https://mascherari.press/onionscan-report-june-2016/>
- [17] Daniela Lopes, Jin-Dong Dong, Pedro Medeiros, Daniel Castro, Diogo Barradas, Bernardo Portela, João Vinagre, Bernardo Ferreira, Nicolas Christin, and Nuno Santos. 2024. Flow Correlation Attacks on Tor Onion Service Sessions with Sliding Subset Sum. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS 2024)*. The Internet Society, San Diego, California, USA, 20 pages. doi:10.14722/ndss.2024.24337
- [18] Srdjan Matic, Platon Kotzias, and Juan Caballero. 2015. CARONTE: Detecting Location Leaks for Deanonymizing Tor Hidden Services. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*. ACM, Denver, Colorado, USA, 1455–1466. doi:10.1145/2810103.2813667
- [19] Nginx. 2026. NGINX: Advanced Load Balancer, Web Server, and Reverse Proxy. <https://nginx.org/>
- [20] Kris Oosthoek, Mark van Staalduinen, and Georgios Smaragdakis. 2023. Quantifying Dark Web Shops' Illicit Revenue. *IEEE Access* 11 (January 2023), 2169–3536. <https://ieeexplore.ieee.org/document/10012354>
- [21] Riseup Collective. 2016. *Best Practices for Hosting Onion Services – riseup.net*. Retrieved June 13, 2025 from <https://help.riseup.net/en/security/network-security/tor/onionservices-best-practices>
- [22] ServerPilot. n.d.. *Enable the Apache Server Status Module*. Retrieved June 12, 2025 from <https://serverpilot.io/docs/guides/apache/status-page/>
- [23] The Hacker News. 2016. *Default Apache Configuration Can Unmask Tor Hidden Services*. Retrieved June 25, 2025 from <https://thehackernews.com/2016/02/apache-tor-service-unmask.html>
- [24] The Tor Project. 2020. *Set up Your Onion Service*. Retrieved July 12, 2026 from <https://community.torproject.org/onion-services/setup/>
- [25] The Tor Project. n.d.. *The Onion Router | Privacy & Freedom Online*. Retrieved June 6, 2025 from <https://www.torproject.org>
- [26] Pascal Tippe and Adrian Tippe. 2024. Onion Services in the Wild: A Study of Deanonymization Attacks. *Proc. Priv. Enhancing Technol.* 2024, 4 (2024), 291–310. doi:10.56553/POPETS-2024-0117

A Ethical Considerations

This work analyses technical metadata from Apache server-status pages collected by the CFLW Dark Web Monitor as part of repeated observations of onion services. The analysis did not require interacting with live child sexual abuse material, creating accounts,

downloading hosted material, or probing services beyond the diagnostic endpoint already present in the collected dataset.

We limited the analysis to metadata needed to understand the exposure and its investigative value, such as server-status fields, timestamps, hostnames, and IP-address fields. We treat these values as infrastructure leads rather than as evidence of operator identity. Where examples are shown, we redact onion addresses, full provider hostnames, unique identifiers, and other details that could unnecessarily expose sensitive infrastructure or aid misuse.

We did not attempt to identify visitors represented in the Client or Request fields. These values are reported only in aggregate, and no raw client IP addresses are published.

The takedown case is described using public law-enforcement reporting together with redacted technical observations. Sensitive case-specific details are omitted unless they are already public or cleared for publication. The goal is to explain the defensive and investigative implications of the localhost-forwarding pitfall without enabling access to illicit services or revealing unnecessary operational details.

Publishing this work may help malicious onion-service operators identify and correct this exposure, reducing its future investigative value. However, the underlying issue is not new and has already been documented in prior work and operational guidance. At the same time, legitimate privacy-sensitive service platforms also need to be aware of this configuration pitfall, as it may unintentionally expose their infrastructure. We therefore consider publication justified because it helps operators recognize and mitigate a real privacy risk.